

slide 4

quiz de abertura

1. python: -4, 512
2. java: 3347
3. javascript: 3, 52
4. c: 0
5. python: true
6. javascript: true, true, true

slide 5

expressões aritméticas

- são formadas por operadores (unários, binários ou ternários, operandos), parênteses e chamadas de função
- exigem duas etapas: obter os operandos, executar as operações

regras de procedência:

- '*' e '/' vem antes, sempre da direita para esquerda (somente convenção de projeto, não uma lei - por precaução sempre usar parênteses)

regras de associatividade:

- quando dois operadores que estão lado a lado, é decidido qual é avaliado primeiro
- quase tudo a esquerda é avaliado primeiro (com exceção de exponenciação)
- Direita: ** (Python, JS, Ruby, Fortran), ++, --, menos unário

ordem de operandos: (slide 6)

- quando não há efeitos colaterais, a ordem de avaliação dos operandos é irrelevante
- java: a ordem é garantida: da esquerda para a direita (p. 308)
- c: a ordem não é definida pelo padrão
- go: a especificação não define a leitura de a em relação à chamada

efeitos colaterais: (slide 7)

- função altera um parâmetro ou uma variável global
- Duas soluções possíveis (p. 308): proibir efeitos colaterais funcionais (difícil em linguagens imperativas) ou fixar a ordem (atrapalha otimizações). Não há solução perfeita. Peguei das notas do autor do slide.

sobrecarregar operadores: (slide 8)

- sobrecarga de operadores: o mesmo símbolo pode ter significados diferentes dependendo do tipo de dado.
- o uso múltiplo de um operador é aceitável desde que não prejudique a legibilidade nem a confiabilidade

mistura de tipos: (slide 9)

- estreitamento: converte para um tipo que não comporta todos os valores do original
- alargamento: converte para um tipo que aloca mais memória
 - normalmente seguro, mas pode perder precisão
- coerção: conversão feita pelo compilador
- expressão de modo misto: operandos de tipos diferentes

coerção implícita: (slide 10)

- coerção implícita acontece quando a linguagem converte automaticamente um tipo de dado em outro durante uma operação ou comparação.
- o custo da coerção é a perda de detecção de erros.

expressões relacionais e booleanas: (slide 11)

- os operadores relacionais têm precedência menor que os aritméticos. o resultado é booleano, exceto que não tenha tipo bool

avaliação em curto-circuito: (slide 12)

- o resultado é determinado sem avaliar todos os operandos
- cuidado com efeitos colaterais: em $(a > b) \parallel (b++ / 3)$, o $b++$ só acontece quando $a \leq b$. Em Python, 0 or "padrão" devolve "padrão": and e or devolvem um dos operandos.

sentenças de atribuição: (slide 13)

- Simples: quase todas as linguagens usam =. ALGOL usou := para não confundir com a igualdade.
- - Composta: $a += b$ (vem do ALGOL 68 e foi adotada por C).
- - Unária: $count++$ e $++count$. Em $- count++$, o incremento acontece antes da negação.

atribuição de modo: (slide 14)

- atribuição de modo misto = tipos diferentes entre variável e expressão.
- saber quando a conversão acontece, porque ela pode ocorrer tarde demais.

estruturas de controle: (slide 15)

- uma estrutura de controle é uma sentença de controle mais as sentenças cuja execução ela controla.
- as computações imperativas avaliam expressões e atribuem valores, mas programas úteis precisam de mais dois mecanismos: SELECIONAR entre caminhos e REPETIR sentenças.
- provaram que qualquer algoritmo representável em fluxograma pode ser escrito só com sequência, seleção de dois caminhos e laço com pré-teste. O goto é desnecessário. Esse resultado apoiou a programação estruturada.

seleção de dois caminhos:

- Seleção de dois caminhos = escolher entre if e else.
 - O else geralmente pertence ao if não pareado mais próximo.
 - Chaves eliminam a ambiguidade.
 - Em algumas linguagens, if também pode retornar um valor.

seleção múltipla: (slide 17)

- Seleção múltipla = escolher entre vários caminhos.
- switch/case é o exemplo clássico.
- C: cuidado com o break e o fall-through.
- C#: não permite queda implícita.
- Go: não cai no próximo caso automaticamente.
- Rust match: deve ser exaustivo.

- Java moderno: switch pode ser expressão e usar ->.

laços controlados por contador: (slide 18)

- Laço controlado por contador: a repetição é determinada por uma variável de controle.

laços lógicos, saídas e iteradores: (slide 19)

- while condição antes → pode executar 0 vezes
- do-while condição depois → executa pelo menos 1 vez
- break sai do laço
- continue vai para a próxima iteração
- for...in JS percorre chaves/índices
- for...of JS percorre valores
- Invariante propriedade que deve continuar válida durante as iterações
- Laço infinito condição nunca se torna falsa

desvio incondicional: (slide 20)

- goto = salto direto para outro ponto do código.
- o goto é a sentença de controle mais poderosa, já que todas as outras podem ser construídas com goto e um seletor. Esse poder é justamente o perigo: sem restrições, o goto torna o programa difícil de ler, pouco confiável e caro de manter. A legibilidade é melhor quando a ordem de execução segue a ordem do texto.

comandos protegidos: (slide 21)

- Todas as guardas são avaliadas
- Várias verdadeiras: escolha não determinística
- Nenhuma verdadeira: erro (no if) ou fim do laço (no do ... od)
- Base do CSP de Hoare (1978) e do select de Go
- Os comandos protegidos são a base do CSP de Hoare (1978), e o select de Go é descendente direto: quando vários canais estão prontos, Go escolhe um ao acaso. Veremos canais na aula de concorrência. As guardas de Haskell (Cap. 15) herdaram a sintaxe, mas são avaliadas em ORDEM (determinístico).

slide 22, 23, 24:

1.
 - a. JavaScript: imprime "Total: 0012". O for...in percorre as chaves (strings), e o + concatena.
 - b. Python: imprime 180.0, mas o esperado era 200.0
2.
 - a. Java: imprime "inválido". O switch sem break cai do case 2 até o default.
 - b. C: imprime "saldo insuficiente" e "fim", mesmo com saque < saldo. O ";" logo depois do if é uma sentença vazia, então o printf fica fora do if.
3.
 - a. Go: imprime "dentro: 20" e "fora: 10". O := dentro do if cria uma NOVA variável x (sombreamento) em vez de atribuir à de fora.
 - b. Java: imprime "0.0%". Divisão inteira 7 / 10 = 0, feita ANTES do alargamento para double (atribuição de modo misto).